

Htet Khant Linn

STAT303 - Mid-Term

March 12, 2026

Install the necessary Library

```
In [18]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
from statsmodels.tsa.arima_process import arma_generate_sample
from PythonTsa.plot_acf_pacf import acf_pacf_fig
from statsmodels.tsa.arima.model import ARIMA
from PythonTsa.LjungBoxtest import plot_LB_pvalue
from scipy import stats
```

1) Select the column from the dataset.

Load Data and Select Column H

```
In [19]: dataset = pd.read_csv("data/arma_dataset_mid_term.csv", index_col = 0, parse_dates=True)
print(dataset.info())
```

```
<class 'pandas.DataFrame'>
DatetimeIndex: 365 entries, 2026-01-01 to 2026-12-31
Data columns (total 15 columns):
 #   Column  Non-Null Count  Dtype  
---  -
 0    1      365 non-null    float64
 1    2      365 non-null    float64
 2    3      365 non-null    float64
 3    4      365 non-null    float64
 4    5      365 non-null    float64
 5    6      365 non-null    float64
 6    7      365 non-null    float64
 7    8      365 non-null    float64
 8    9      365 non-null    float64
 9   10      365 non-null    float64
10   11      365 non-null    float64
11   12      365 non-null    float64
12   13      365 non-null    float64
13   14      365 non-null    float64
14   15      365 non-null    float64
dtypes: float64(15)
memory usage: 45.6 KB
None
```

```
In [20]: print(dataset.head())
```

	1	2	3	4	5	6	\
2026-01-01	0.496714	-0.480648	0.534629	0.195845	-0.579124	-0.285094	
2026-01-02	0.159764	1.530021	-2.154582	-0.860866	-1.002045	-0.460053	
2026-01-03	0.743547	3.966219	-2.128214	-0.108267	0.292374	-1.581832	
2026-01-04	1.969158	4.003751	-0.734936	-1.767544	0.303742	-3.666223	
2026-01-05	0.947341	3.189970	0.687931	-0.031371	-0.370243	-5.253175	

	7	8	9	10	11	12	\
2026-01-01	-0.138456	1.503420	0.441941	-0.718832	-0.401220	-0.080717	
2026-01-02	-1.348909	1.022958	-0.692652	0.098056	-0.016640	0.030205	
2026-01-03	-1.373197	-0.583946	0.628447	0.511749	0.002609	-1.980078	
2026-01-04	-1.620728	-1.245237	0.780008	-0.064610	0.099241	-0.271719	
2026-01-05	-1.731096	-0.944234	0.369418	-0.080813	-0.713465	0.183457	

	13	14	15
2026-01-01	-1.598124	0.408053	0.157128
2026-01-02	-0.976139	0.918045	0.506158
2026-01-03	1.721109	0.920089	0.136823
2026-01-04	0.307104	-0.573273	-0.220286
2026-01-05	-0.202011	-0.832516	-0.943065

```
In [21]: y = dataset['7']
print('--- My Dataset Head ---')
print(y.head())
print()

print('--- My Dataset Length ---')
print(len(y))
```

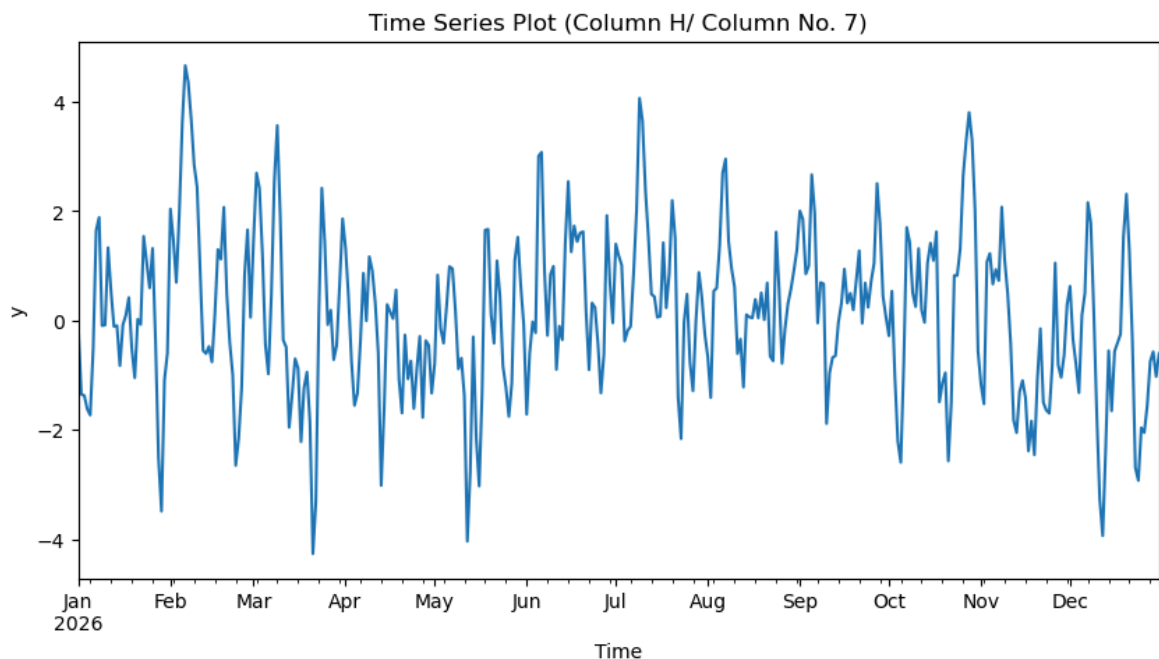
```
--- My Dataset Head ---
2026-01-01    -0.138456
2026-01-02    -1.348909
2026-01-03    -1.373197
2026-01-04    -1.620728
2026-01-05    -1.731096
Name: 7, dtype: float64
```

```
--- My Dataset Length ---
365
```

2) Plot the figure and name the y-axis as y and the x-axis as time, and the title is Time Series Plot.

```
In [22]: y.plot(figsize=(10,5))
plt.title("Time Series Plot (Column H/ Column No. 7)")
plt.xlabel('Time')
plt.ylabel('y')

plt.savefig('output/time_series_plot')
plt.show()
```



3) If the time series has a trend, you need to use `diff()` to make it stationary; if not, you can skip to step 4.

5) Test the stationarity using the KPSS

Test for Stationarity Using KPSS

The output of the KPSS test contains 4 things:

- The KPSS statistic
- p-value
- Number of lags used by the test
- Critical values

Source - [Click Here](#)

```
In [23]: result = sm.tsa.stattools.kpss(y, regression='c', nlags=20)
print(f"KPSS Statistic: {result[0]}")
print(f"p-value: {result[1]}")
print(f"Lags: {result[2]}")
print(f"Critical Values: {result[3]}")
```

KPSS Statistic: 0.15808021366600225

p-value: 0.1

Lags: 20

Critical Values: {'10%': 0.347, '5%': 0.463, '2.5%': 0.574, '1%': 0.739}

C:\Users\Charles_the_king\AppData\Local\Temp\ipykernel_14252\2909912194.py:1: InterpolationWarning: The test statistic is outside of the range of p-values available in the look-up table. The actual p-value is greater than the p-value returned.

```
result = sm.tsa.stattools.kpss(y, regression='c', nlags=20)
```

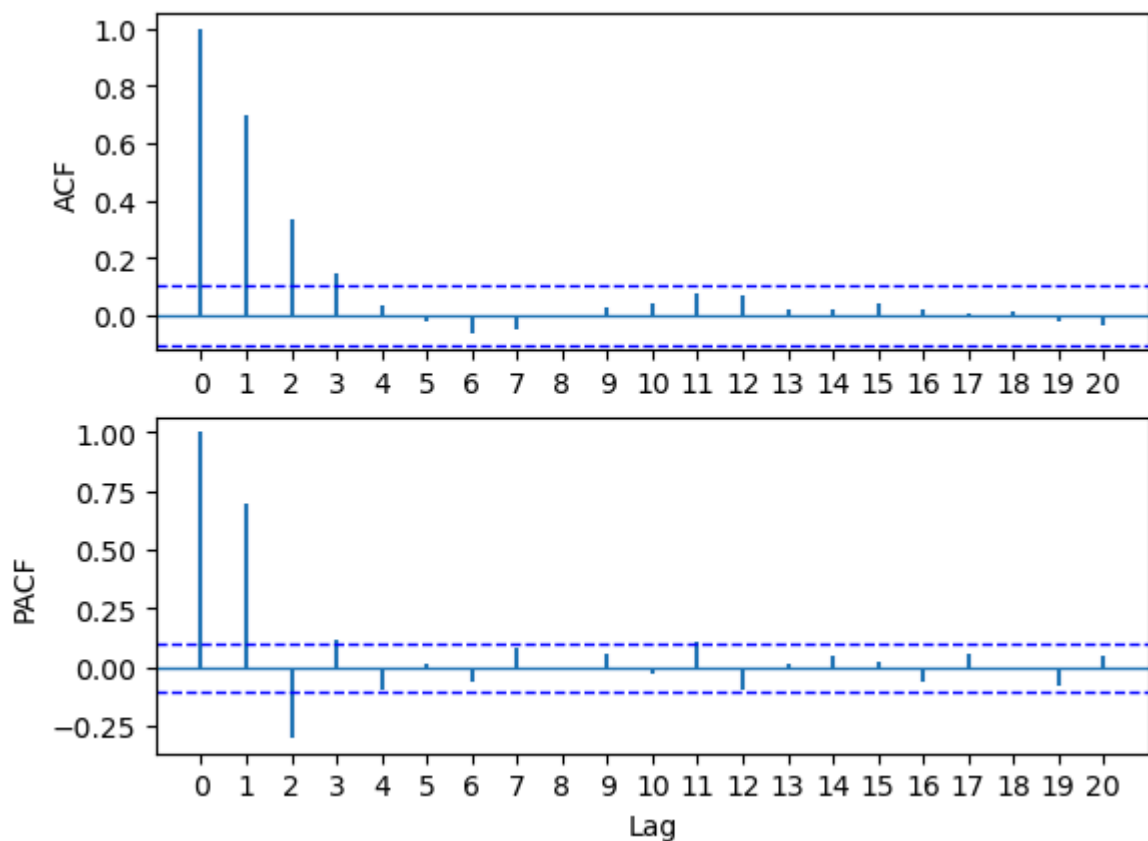
Since the p-value is greater than 0.05, we can assume the data is **stationary** and proceed with $d=0$.

4) Plot the ACF and PACF using this time series.

```
In [24]: acf_pacf_fig(y, both=True, lag=20)

plt.savefig('output/acf_pacf.png')

plt.show()
```



6) Choose the order of AR and MA

```
In [25]: inf = sm.tsa.arma_order_select_ic(y, max_ar=4, max_ma=4, ic=['aic', 'bic', 'hqic'])
```

```
c:\Users\Charles_the_king\miniconda3\envs\STAT303\Lib\site-packages\statsmodels\t
sa\statespace\sarimax.py:966: UserWarning: Non-stationary starting autoregressive
parameters found. Using zeros as starting parameters.
    warn('Non-stationary starting autoregressive parameters')
c:\Users\Charles_the_king\miniconda3\envs\STAT303\Lib\site-packages\statsmodels\t
sa\statespace\sarimax.py:978: UserWarning: Non-invertible starting MA parameters
found. Using zeros as starting parameters.
    warn('Non-invertible starting MA parameters found.')
c:\Users\Charles_the_king\miniconda3\envs\STAT303\Lib\site-packages\statsmodels\b
ase\model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to c
onverge. Check mle_retvals
    warnings.warn("Maximum Likelihood optimization failed to ")
c:\Users\Charles_the_king\miniconda3\envs\STAT303\Lib\site-packages\statsmodels\b
ase\model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to c
onverge. Check mle_retvals
    warnings.warn("Maximum Likelihood optimization failed to ")
```

```
In [26]: print("Best AIC order:", inf.aic_min_order) # Shows best model by AIC
```

```
Best AIC order: (np.int64(3), np.int64(1))
```

```
In [27]: print("Best BIC order:", inf.bic_min_order) # Shows best model by BIC
```

```
Best BIC order: (np.int64(1), np.int64(1))
```

```
In [28]: print("Best HQIC order:", inf.hqic_min_order) # Shows best model by HQIC
```

```
Best HQIC order: (np.int64(1), np.int64(1))
```

7) Run the ARIMA model using the order that you choose.

```
In [29]: arima11 = ARIMA(y, order=(1,0,1), trend='c').fit()
print(arima11.summary())
```

```
c:\Users\Charles_the_king\miniconda3\envs\STAT303\Lib\site-packages\statsmodels\t
sa\base\tsa_model.py:473: ValueWarning: No frequency information was provided, so
inferred frequency D will be used.
    self._init_dates(dates, freq)
c:\Users\Charles_the_king\miniconda3\envs\STAT303\Lib\site-packages\statsmodels\t
sa\base\tsa_model.py:473: ValueWarning: No frequency information was provided, so
inferred frequency D will be used.
    self._init_dates(dates, freq)
c:\Users\Charles_the_king\miniconda3\envs\STAT303\Lib\site-packages\statsmodels\t
sa\base\tsa_model.py:473: ValueWarning: No frequency information was provided, so
inferred frequency D will be used.
    self._init_dates(dates, freq)
```

SARIMAX Results

```

=====
Dep. Variable:          7    No. Observations:          365
Model:                 ARIMA(1, 0, 1)    Log Likelihood    -516.037
Date:                 Thu, 12 Mar 2026    AIC              1040.073
Time:                 21:53:09    BIC              1055.673
Sample:              01-01-2026    HQIC             1046.273
                  - 12-31-2026
Covariance Type:      opg
=====

```

	coef	std err	z	P> z	[0.025	0.975]
const	0.1010	0.147	0.687	0.492	-0.187	0.389
ar.L1	0.4690	0.063	7.472	0.000	0.346	0.592
ma.L1	0.4980	0.064	7.751	0.000	0.372	0.624
sigma2	0.9872	0.074	13.373	0.000	0.843	1.132

```

=====
==
Ljung-Box (L1) (Q):          0.01    Jarque-Bera (JB):          0.
01
Prob(Q):                    0.94    Prob(JB):                  1.
00
Heteroskedasticity (H):      0.92    Skew:                      -0.
00
Prob(H) (two-sided):         0.65    Kurtosis:                  3.
02
=====
==

```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

8) Test with normaltest() using the residuals.

Reference - [normaltest](#)

```

In [30]: resid11 = arima11.resid

norm_test = stats.normaltest(resid11)
print(f"Normality test statistic: {norm_test[0]}")
print(f"p-value: {norm_test[1]}")

```

Normality test statistic: 0.06446943472258876
p-value: 0.9682792834463669

This function (normaltest()) tests the null hypothesis that a sample comes from a normal distribution. The p-value is 0.9683, which is much greater than 0.05. p-value > 0.05 means we FAIL to reject the null hypothesis. The data comes from a normal distribution, showing that my residuals are normally distributed. This shows that my model is good.

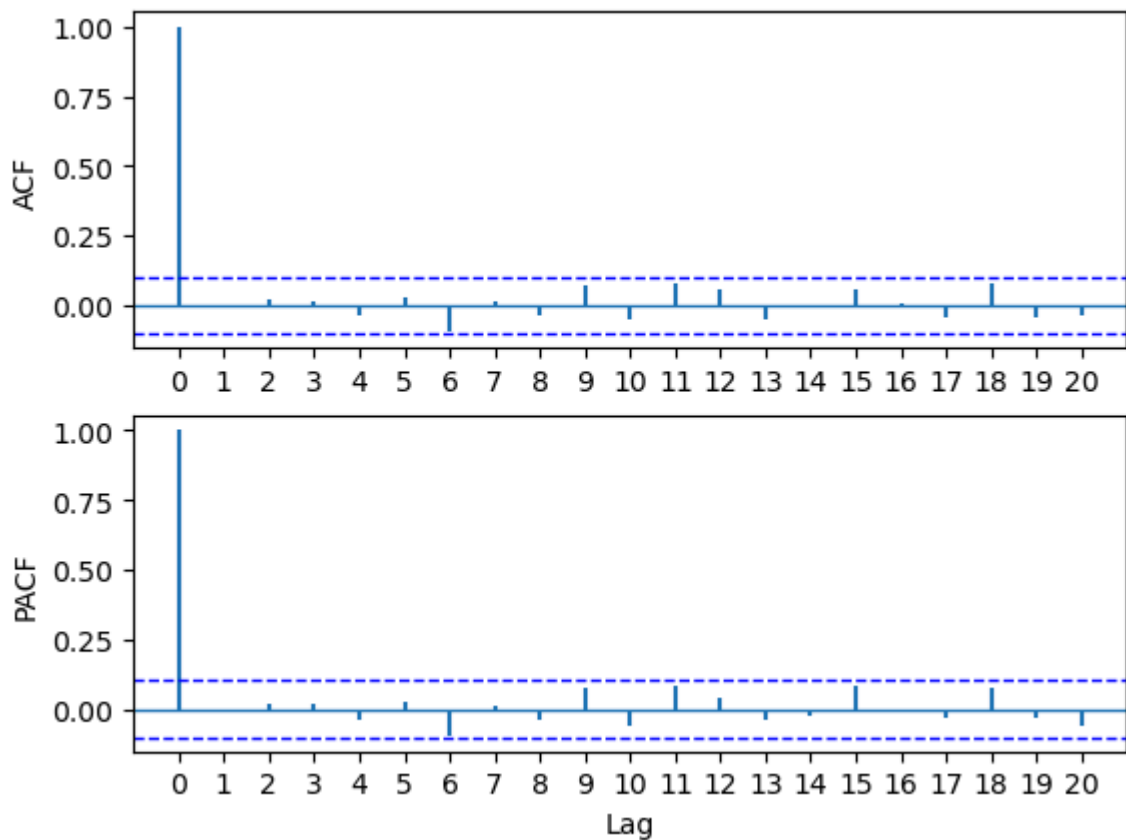
9) Plot the ACF and PACF again using the residuals.

```

In [31]: acf_pacf_fig(resid11, both=True, lag=20)

```

```
plt.savefig("output/residuals_acf_pacf.png")
```

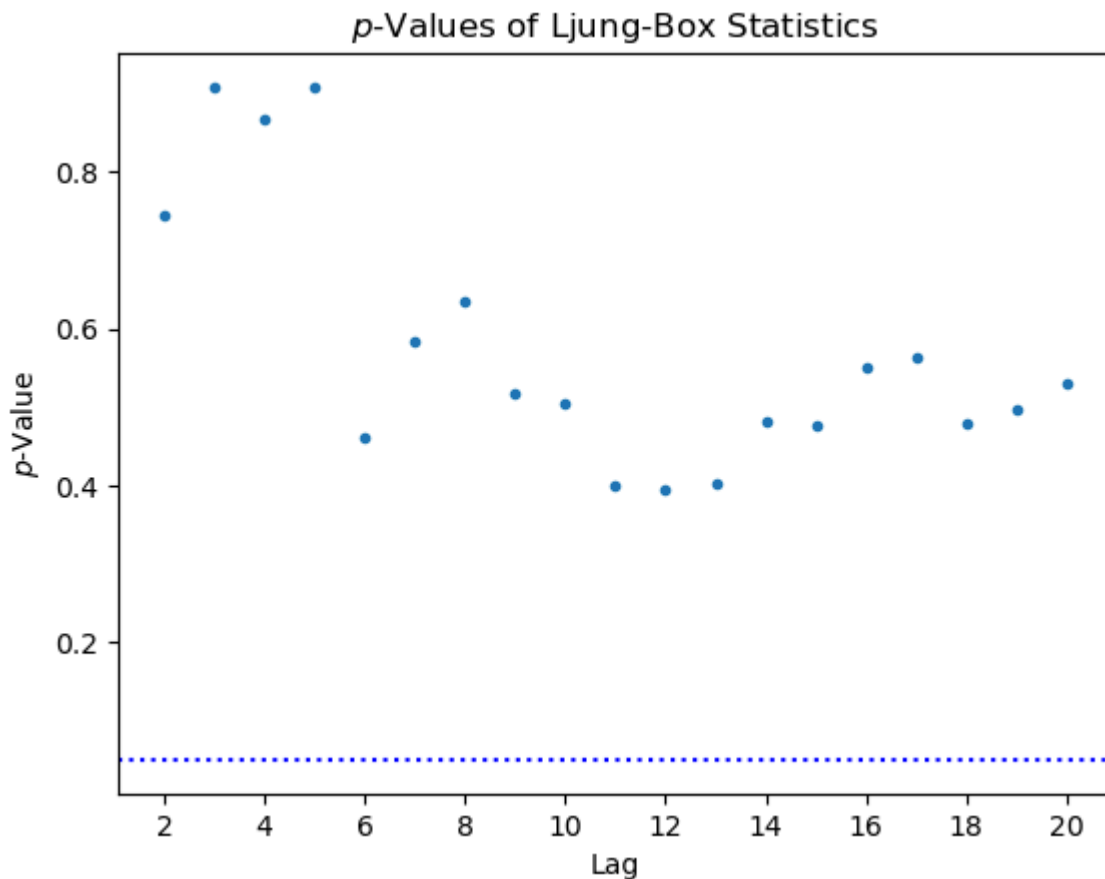


10) Plot the Ljung-Box statistic plot using the residuals.

```
In [32]: plot_LB_pvalue(resid11, noestimatedcoef=1, nolags=20)

plt.savefig("output/residuals_ljung-box.png")

plt.show()
```



11) Make the prediction with 9 additional points.

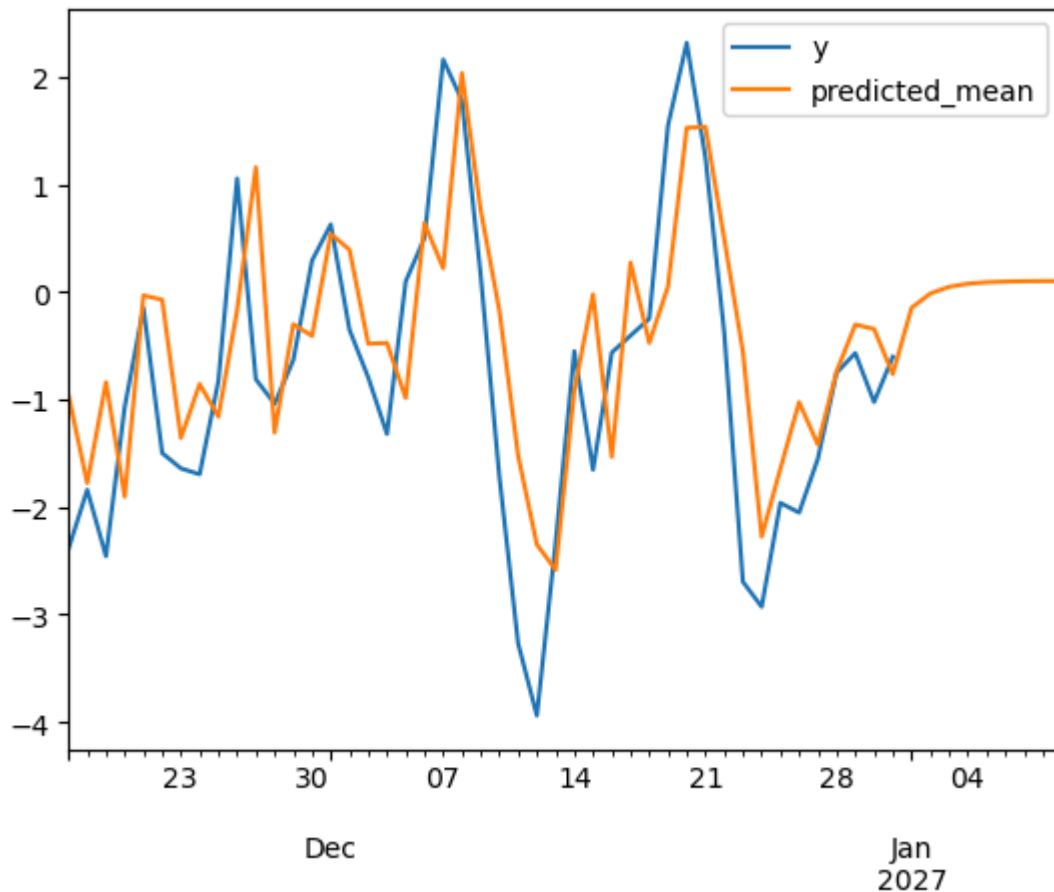
12) Plot the prediction and the y values.

```
In [33]: pred = arima11.get_prediction(start=320, end=373) # data has 365 points with in
predicts = pred.predicted_mean

predframe = pd.concat([y[320:], predicts], axis = 1)
predframe.columns = ['y', 'predicted_mean']

predframe.plot()

plt.savefig('output/prediction.png')
plt.show()
```

13) Write the equation from the summary of the ARIMA model.

$$y_t = 0.1010 + 0.4690 y_{t-1} + \epsilon_t + 0.4980 \epsilon_{t-1}$$